Tool Use I: From Text Generator to Actor

FRIDAY AI Education — Episode 034 · 2026-07-20 · Printable Companion

FRIDAY AI Education — Episode 034 · Printable Companion Paper

Unit: Tool Use, Part 1 of 3 · Lesson 20 of 37 in the AI-Native Operator arc

Abstract

A language model, on its own, produces text and nothing else. It cannot read a file, send an email, or move money. Tool use is the mechanism that changes this: the model is given a menu of functions it may request, it emits structured requests, and a surrounding program executes them and reports back. This paper explains that mechanism precisely, then builds the distinction that everything in agent design rests on — the line between actions that *observe* the world and actions that *change* it. Reading a file and editing a file look almost identical in a transcript; they are categorically different events. One is a question. The other is a side effect: a change to external state that persists after the conversation ends. Understanding where that line sits, and why the model itself cannot see it, is the prerequisite for the next two lessons on permissions and approval. It is also the difference between an operator who deploys agents deliberately and one who discovers, after the fact, what their system was allowed to do.

1. Where This Sits

The last three lessons were about workflow specs: defining an objective, an input and output contract, gates, failure handling, and a concrete definition of done. A spec describes what a workflow *should* do. This lesson is about the machinery through which an AI system *does* anything at all.

That machinery is tool use. It is the single idea that separates a chatbot from an operator. Before tool use, the worst outcome of a bad model response was bad text — wrong, embarrassing, misleading, but inert. After tool use, a bad response can be a sent email, a deleted record, a submitted form. The spec vocabulary you already have — gates, unsafe action, recovery — was built in anticipation of exactly this moment. Today we look at what those gates are actually gating.

One boundary to draw before we start: this lesson is about *how* a model acts. The next lesson, Tool Use II, is about *whether it should be allowed to* — permissions, least privilege, and approval. Keep those separate. You cannot design good permissions for a mechanism you don't yet understand.

2. The Model Alone: A Text-In, Text-Out Machine

Start from a fact that is easy to state and easy to forget: a language model is a function from text to text. Tokens in, tokens out. It has no network access, no filesystem, no hands. When a model "browses the web" or "checks your calendar," the model is not doing those things. Something else is.

This is not a technicality. It is the architectural truth that makes everything else in this paper legible. The model's only output channel is generated text. If an AI system affects the world, there is always — always — a piece of ordinary software standing between the model's text and the effect. That software reads what the model wrote, decides

whether to act on it, performs the action, and tells the model what happened.

Call that surrounding software the **harness** (you will also see *orchestrator*, *runtime*, *agent loop* — same idea). The harness is not intelligent. It is a loop written by a programmer. But it holds all of the actual power in the system. The model proposes; the harness executes.

Why does this matter to a founder rather than just an engineer? Because it locates responsibility. When people ask "can the AI do X?", the technically correct answer is "the AI can *ask* for X; the question is what the harness lets it have." Every safety property, every permission, every audit log lives in the harness layer. Every capability decision is a harness decision. Vendors who blur this — who talk about the model as if it reaches into the world directly — are either simplifying or selling.

3. The Mechanism: Function Calling

Tool use is implemented through a protocol usually called **function calling**. (OpenAI's documentation, linked in the reading guide, is the canonical description; Anthropic's equivalent is called tool use. The mechanics are the same shape.) It works in five steps.

Step 1 — Declaration. The developer describes a set of functions to the model, in the request itself. Each declaration has a name, a plain-language description, and a schema for its parameters. For example:

```json
{
"name": "send_email",
"description": "Send an email from the founder's account.",
"parameters": {
"to": "string",
"subject": "string",
"body": "string"
}
}
```

Note what this is: text. The model does not gain an ability here; it gains *vocabulary*. It now knows that a thing called `send_email` exists and what a well-formed request for it looks like.

Step 2 — The model requests a call. When the model decides a tool would help — because the user asked for something the tool covers, or because its instructions steer it there — it stops producing prose and instead emits a structured request: the function name plus arguments matching the schema. Crucially, this is still just generated text in a special format. The model has predicted that `send_email({"to": "anna@fund.com", ...})` is the right next output, the same way it predicts any other token sequence.

Step 3 — The harness executes. The harness parses the request, validates it, and — if its rules allow — runs the actual code: calls the email API, queries the database, hits the search endpoint. This step is deterministic software. The model is not involved and is, in fact, frozen mid-conversation, waiting.

Step 4 — The result returns as text. Whatever the tool produced — search results, file contents, an error message, a confirmation — is serialized and appended to the conversation as a new message. From the model's perspective, tool results are just more context: text that arrived, exactly like a user message.

Step 5 — The loop continues. The model reads the result and decides what's next: answer the user, call another tool, or recover from an error. A multi-step task is just this loop running several times. What people call an "agent" is, at the mechanical level, this loop plus a goal.

Three consequences of this design are worth pinning down, because they carry the rest of the lesson:

1. **The model never verifies its own actions directly.** It knows only what the harness reports. If the harness says the email sent, the model believes the email sent. Its picture of the world is entirely mediated.
2. **Tool selection is prediction, not deliberation.** The model chooses tools the same way it chooses words — by likelihood given context. Usually that's right. Sometimes it calls the wrong tool, or the right tool with invented arguments, with the same fluent confidence either way. You already know from earlier lessons that a model's confidence is not evidence; that fact just became operationally expensive.
3. **The tool list is the capability surface.** The model can only request what it has been told exists. This is the design lever everything in Tool Use II hangs on: you shape what a system can do by shaping what you declare to it.

4. The New Concept: Side Effects and State Change

Here is today's genuinely new layer. Tools are not all the same kind of thing, and the difference that matters is not what domain they touch — files, email, web — but whether they *change* anything.

Borrow two terms from software engineering, where they have precise meanings:

A **side effect** is any change a function makes beyond returning a value. A function that takes a number and returns its square has no side effects; run it a thousand times and the world is identical. A function that writes to a file, sends a message, or updates a database has side effects: the world is different after it runs.

State is the persistent condition of a system — the contents of your files, the rows in your CRM, the emails in someone's inbox, your balance at the bank. A **state change** is a side effect that persists: the world stays different after the conversation ends, after the process exits, after everyone has forgotten why.

Now apply this to tools. Every tool falls on one side of a line:

Read tools observe state without changing it. Fetch a file's contents, run a search, query a database, look at a web page. Their defining property is *repeatability without consequence*: call them once or fifty times and the world is the same. The worst realistic outcome of a bad read is bad information — the model retrieves the wrong document and reasons from it. That's a real failure, but it stays inside the conversation, and the remedy is cheap: read again, read better.

Write tools change state. Edit the file, send the message, insert the row, submit the form, charge the card. Their defining property is that *the action outlives the conversation*. The failure now escapes into the world, where it must be found and undone — if it can be undone at all.

The read/write line matters more than any other classification of tools because it determines what a mistake *costs*. And within writes, there is a second gradient worth naming: **reversibility**. Editing a file under version control is a write

you can undo in seconds. Updating a CRM record is a write you can undo if you notice. Sending an email is a write you cannot undo at all — you can only send a second email, which is not the same thing. As reversibility falls, the price of a model error rises from "annoying" toward "permanent." Wire transfers, published posts, and messages to other humans sit at the expensive end.

One refinement, so the categories don't oversimplify: some reads have side effects on the *other* side. Fetching a web page can fire analytics; polling an API consumes rate limits; a search query is logged somewhere. The clean question is not "is this labeled a read?" but "**what is different in the world after this runs, and who would care?**" That question, asked of every tool in a system, is the whole discipline in one sentence.

5. Three Pairs That Make the Line Concrete

The abstraction lands fastest through pairs of tools that look like siblings and are not.

5.1 File read versus file edit

`read_file` returns the contents of a document. `edit_file` replaces some of them. In a transcript of an agent session, these look nearly identical — two tool calls, two results, a few lines apart. But consider each one's failure mode. A wrong read means the model saw stale or irrelevant content; the damage is confined to this conversation's quality. A wrong edit means a document on disk is now silently wrong — a financial model with a bad cell, a contract clause altered, a config value changed — and every future reader, human or machine, inherits the error. The read failure decays when the conversation ends. The write failure compounds.

5.2 Search versus send

`search_email` and `send_email` might live in the same integration, share an authentication token, and appear adjacent in the tool list. One inspects an inbox; the other performs a speech act on your behalf to another human being. A botched search wastes a turn. A botched send is an event in a relationship: the half-drafted note to an investor, the reply-all, the message to the wrong Anna. Sent email is the canonical irreversible write — there is no undo, only damage control. That these two operations so often ship in the same bundle, guarded by the same credential, is exactly the problem the next lesson exists to solve.

5.3 The browser form

The browser is the instructive edge case because it holds both categories behind one interface, separated by a single click. An agent that *reads* a form — navigating to a checkout page, examining the fields, extracting the price — is observing. The moment it fills the fields and presses **Submit**, it has crossed the line: an order placed, an account created, a payment initiated on someone else's servers, where your undo button does not reach. The web's own architecture encodes this distinction — a GET request is meant to be a safe observation, a POST is a submission that changes something — and browsing agents inherit it whether their designers thought about it or not. The lesson generalizes: interfaces built for humans mix reads and writes freely, because a human at the keyboard was assumed to know which was which. An agent driving those interfaces has no such instinct unless you build it one.

5.4 A compact map

Tool call	Class	Reversible?	Worst realistic failure	Where the failure lives
Read a file	Read	— (nothing to reverse)	Model reasons from wrong content	Inside the conversation

Tool call	Class	Reversible?	Worst realistic failure	Where the failure lives
Edit a file	Write	Usually (with version control)	Silent corruption of a document others rely on	Your filesystem, downstream readers
Search an inbox	Read	—	Wasted turn, wrong context	Inside the conversation
Send an email	Write	**No**	Wrong message to a real person	A human relationship
View a web form	Read	—	Misread the page	Inside the conversation
Submit a web form	Write	Rarely	Order, signup, or payment on external systems	Someone else's servers
Query a database	Read	—	Stale or wrong rows retrieved	Inside the conversation
Update / delete a row	Write	Sometimes (backups, soft delete)	Record loss or corruption at scale	Your system of record

Read down the "where the failure lives" column. That column is the entire argument of this paper in one place: reads fail *inward*, writes fail *outward*.

6. The Model Cannot See the Line

Here is the uncomfortable part, and the reason this lesson precedes rather than follows the one on permissions.

To the model, `read_file` and `send_email` are structurally identical: a name, a description, a schema. The read/write distinction we just spent three sections building is *not represented anywhere in the protocol*. Nothing in a function declaration marks it as irreversible. The model may infer consequence from a tool's name and description — good models often do — but that inference is probabilistic, exactly like everything else the model does. It is a tendency, not a guarantee, and tendencies are not what you build financial controls on.

So the system's judgment about consequence has to live where judgment can be made reliable: in the harness, and in the humans who configure it. Someone must decide, tool by tool, *this one is safe to call freely; this one needs a confirmation; this one requires a human*. That decision cannot be delegated to the model, for the same reason a company doesn't let an employee set their own spending authority — not because the employee is malicious, but because the incentive structure is wrong and the failure is unbounded.

A second consequence follows from Section 3's mechanics: because the model only knows what the harness reports, error handling around writes is where agent systems earn or lose their keep. A read that fails and silently returns nothing produces a confused model. A *write* that fails while the model believes it succeeded — or succeeds while the model believes it failed and therefore retries — produces duplicated emails, double-charged cards, and records edited twice. Idempotency (the property that repeating an action once more changes nothing further) is a dull word that prevents expensive incidents. Notice that your workflow-spec vocabulary from lessons 18–19 was quietly preparing for this: an output contract for a workflow with write tools is largely a contract about *which state changes are permitted and how you verify they happened once*.

7. The Operator's View: Capability Surface as an Org Design Decision

Step back from mechanism to company-building, because this is where the concept pays rent.

When you connect a model to tools, you are making the same category of decision as when you grant an employee system access — with one important difference: the "employee" executes at machine speed, does not get nervous before doing something consequential, and will exhibit the same failure with perfect consistency until someone notices. The hiring analogy is imperfect (agents are not employees, magic or otherwise), but the *access control* analogy is exact. Nobody gives a new hire the production database password and the company credit card on day one. The tool list is where that instinct must be applied to AI systems — and where, in practice, it often isn't, because integrations arrive in bundles and the send capability rides in with the search capability.

A realistic scenario. Suppose you're deploying an assistant for customer operations at a small fintech — a Galoy-like system, say. The proposed tool list: look up a customer's account, search the support history, draft a reply, send the reply, and issue a refund up to some limit. Run each tool through this lesson's lens. *Look up* and *search* are reads: failures stay inward, so let the model call them freely — generous read access is usually cheap and makes the model smarter. *Draft* is interesting: it produces text but changes no external state, so it's effectively a read-class operation — which suggests a clean design where the model drafts and a human sends. *Send* is an irreversible write to a customer relationship. *Refund* is a write to money. The right deployment question is not "is the model good enough to have these tools?" — a question about intelligence — but "for each write, what does a mistake cost, who catches it, and how is it undone?" — a question about consequence. Operators who ask the second question ship agents that survive contact with reality. Those who ask only the first ship demos.

The FRIDAY relevance, stated once and briefly: GBrain-style knowledge work — searching a corpus, retrieving, citing, summarizing — lives almost entirely on the read side, which is precisely why a public-content knowledge base is a sound *first* wedge: maximal usefulness, minimal blast radius. The moment FRIDAY graduates from answering questions to filing, sending, and updating, it crosses the line this paper draws, and everything from lessons 17–19 — gates, failure handling, done-conditions — stops being paperwork and becomes the control system for real state changes. That is the trajectory, and it should be walked in that order.

8. Limitations and Failure Modes

Four things this lesson's machinery does not solve, so you don't over-trust it.

Schemas validate form, not sense. The harness can verify that `send_email` received a string in the `to` field. It cannot verify that the string is the right recipient, that the amount in a refund call matches the customer's actual claim, or that the file being edited is the one the user meant. A perfectly well-formed call can be perfectly wrong. Structured output is often mistaken for verified output; it is nothing of the kind.

Tool descriptions are instructions the model may misread. The model's understanding of what a tool does comes entirely from its name and description — prose, written by a developer, interpreted probabilistically. Ambiguous descriptions produce wrong calls. This also opens a genuine security surface: text the model reads (a web page, a retrieved document, an email) can attempt to steer its tool calls. When the reader of untrusted text holds write tools, that text becomes a potential command channel. Treat this as a named hazard for now — *prompt injection* — to be handled properly in the permissions lesson.

The loop amplifies. A single wrong call is one incident. An agent loop retries, chains, and compounds: a model that misdiagnoses a failed write may "fix" it four more times before anyone looks. Speed and persistence, the loop's virtues, are exactly what convert a small error into a large one.

Observation is only as honest as the harness. The model's world-model is the transcript. If tool results are wrong, truncated, or lie by omission, the model reasons flawlessly from a false premise. Auditing an agent system therefore

means auditing what it was *told*, not just what it *did*.

None of these are arguments against tool use. They are the precise reasons the next lesson exists: permissions, least privilege, and approval are the countermeasures matched to these failure modes, one by one.

9. Vocabulary

Tool use — The general mechanism by which a language model participates in actions beyond text generation: it requests operations from a predefined set, and a surrounding program executes them and returns results.

Function calling — The concrete protocol implementing tool use: functions are declared to the model with names, descriptions, and parameter schemas; the model emits structured call requests; the harness executes and returns results as text.

Harness (orchestrator, agent loop) — The ordinary, deterministic software surrounding the model that declares tools, parses the model's requests, enforces rules, executes actions, and reports results. The locus of all real capability and all real control.

Side effect — Any change a function makes to the world beyond returning a value: writing a file, sending a message, updating a record. The property that separates observation from action.

State — The persistent condition of a system: file contents, database rows, inbox contents, account balances.

State change — A side effect that persists beyond the conversation or process that caused it.

Read tool / write tool — A tool that observes state without changing it, versus one that changes it. Read failures stay inside the conversation; write failures escape into the world.

Reversibility — The degree to which a state change can be undone. A version-controlled file edit is highly reversible; a sent email is not. The cost of model error scales with irreversibility.

Idempotency — The property that performing an action again produces no further change. The difference between a harmless retry and a double charge.

Capability surface — The full set of tools declared to a model: the outer boundary of everything the system could possibly do, however unlikely.

10. Reading Guide

OpenAI — Function Calling Guide

<https://platform.openai.com/docs/guides/function-calling>

Read this as the reference implementation of Section 3, not as a tutorial to complete. Suggested approach, twenty minutes:

1. **Read the overview and lifecycle diagram first.** Match each stage to the five steps in Section 3 — declaration, model request, execution, result, loop. The guide's core sentence to internalize is that the model *generates*

function calls; your code *executes* them. That one sentence is this entire paper in the vendor's own words.

2. **Look at one full function definition** (the JSON schema example). Notice that nothing in the format expresses consequence — there is no `irreversible: true` field. The read/write distinction you now carry is invisible to the protocol. Seeing that absence firsthand is the point of the exercise.
3. **Skim the section on handling results and multi-turn loops**, watching for how errors are returned to the model. Ask of it: if the execution failed but the result message were misleading, what would the model believe?
4. **Skip** the SDK-specific plumbing (streaming, parallel calls, language bindings) unless curiosity strikes. It's implementation detail below the altitude of this lesson.

Anthropic's tool-use documentation covers the same protocol with different field names; a skim is optional and will mostly confirm that the shape is industry-wide, not one vendor's design.

11. Walking Questions

Take these on the next walk, or into the voice memo.

1. **Explain the loop.** Without notes: walk through what happens, step by step, between a user typing "email Anna the deck" and the email leaving the server. Where exactly does the model stop and the harness begin? If you can't place that boundary crisply, reread Section 3 — it is the load-bearing wall.
2. **Find the blurry edge.** Which felt less obvious: *why* the model can't see the read/write line (Section 6), or *what to do about it* as an operator (Section 7)? Name the blur precisely; it tells you what the next listen-through should target.
3. **Locate it in FRIDAY.** List, from memory, three operations FRIDAY or GBrain performs today. Classify each as read or write. Is the current system living entirely on the read side — and if so, what is the *first* write it will need, and what would that write cost when it goes wrong?
4. **Design the bounded test.** Pick one real workflow in a company you know — invoice chasing, meeting scheduling, support triage. Draw its tool list, split reads from writes, and identify the single write you would automate first *because* it is the most reversible. That exercise — small on paper, done honestly — is the application artifact for this episode. It should fit on one page.

12. One-Sentence Takeaway

Tool use turns a model's text into requests that other software executes, so the moment you grant a write tool, you have stopped deploying a text generator and started delegating changes to the world — and the model, which cannot tell the difference, will never be the one managing that risk for you.

Next in this unit — Tool Use II: Permissions, Least Privilege, and Approval. Having drawn the line between observing and acting, we design the controls that decide who may cross it, when, and with whose sign-off.